

83. Une « struct » : ce qui va ensemble

Une troupe d'ennemis, et non six tableaux parallèles.

1. Ce qu'il faut comprendre

Un ennemi a une position, une vitesse, une santé. Sans `struct`, cela fait trois tableaux qu'il faut penser à garder alignés — et le jour où l'on en trie un sans trier les autres, les vies changent de propriétaire.

Une `struct` range ces champs ensemble. `Etoile etoiles[4];` réserve quatre enregistrements ; `etoiles[i].x` en désigne un champ. Le compilateur calcule le décalage lui-même, et `sizeof(Etoile)` dit ce qu'un enregistrement pèse.

Le numéro d'un lutin peut être **calculé** : `sprite(i, ...)` dans une boucle affiche toute la troupe, chacun avec ses valeurs à lui. C'est ce qui rend la `struct` utile plutôt que décorative.

Une `struct` peut en contenir une autre : `struct Ennemi { Point ou; uint8_t vie; };`, et l'on écrit `troupe[i].ou.x`. Le compilateur additionne les décalages lui-même.

Ce qu'elle ne sait pas faire : s'affecter d'un bloc, se passer en argument, se rendre par une fonction. On nomme le champ qu'on veut lire ou écrire — et le compilateur le dit plutôt que de traduire à peu près.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **742 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	57 o	← la console, à l'allumage - la mise en route de la console
placer()	120 o	← main() - ta fonction
avancerToutes()	208 o	← main() - ta fonction
main()	28 o	← démarrage - ta fonction
VBlank	8 o	← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage	28 o	← image() - attend l'image suivante
AttendreVBlank	16 o	← démarrage - attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte - attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() - écrit un texte, lettre par lettre
RangerLutins	11 o	← démarrage - range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage - installe la copie des lutins en mémoire rapide
CopierTuiles	67 o	← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage - vide les deux cartes de l'écran
Tuiles	128 o	← CopierTuiles - 7 lettres sur 43, et 1 tuile dessinée
DonneesTransfert	10 o	← InstallerTransfert - la petite routine qui copie les lutins
"UNE TROUPE"	10 o	← texte() - un texte gravé

Lettres gravées : ENOPRTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes

Fonctions du langage appelées :

```
sprite()      × 1    88 o sur place, dans avancerToutes()
texte()       × 1    11 o sur place, dans main() + EcrireTexte (10 o)
image()       × 1     6 o sur place, dans main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
placer()      × 1     3 o sur place, dans main()
avancerToutes() × 1    3 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <Tuile>    // un dessin de 8 × 8 pixels
#include <sprite>   // place un lutin de 8 × 8 au pixel près
#include <texte>    // écrit un texte à l'écran
```

2. UN DESSIN — CORPS

```
Tuile CORPS = {
    "00333300",
    "03222230",
    "32222223",
    "32122123",
    "32222223",
    "32211223",
    "03222230",
    "00333300",
};
```



à l'écran

3. UN ENREGISTREMENT — ETOILE

```
struct Etoile {
    uint8_t x;
    uint8_t y;
    uint8_t vitesse;
};
```

4. UNE CONSTANTE — COMBIEN

```
const uint8_t COMBIEN = 4;
```

5. UN TABLEAU — ETOILES

```
Etoile etoiles[COMBIEN];
```

6. UNE FONCTION — PLACER()

```
void placer() {
    for (uint8_t i = 0; i < COMBIEN; i++) {
        etoiles[i].x = 20 + i * 30;
        etoiles[i].y = 32 + i * 24;
        etoiles[i].vitesse = 1 + i;
    }
}
```

7. UNE FONCTION — AVANCERTOUTES()

```
void avancerToutes() {
    for (uint8_t i = 0; i < COMBIEN; i++) {
        etoiles[i].x += etoiles[i].vitesse;

        if (etoiles[i].x > 152) etoiles[i].x = 0;

        sprite(i, etoiles[i].x, etoiles[i].y, CORPS);
    }
}
```

8. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(2, 1, "UNE TROUPE");

    placer();

    while (true) {
        image();
        avancerToutes();
    }

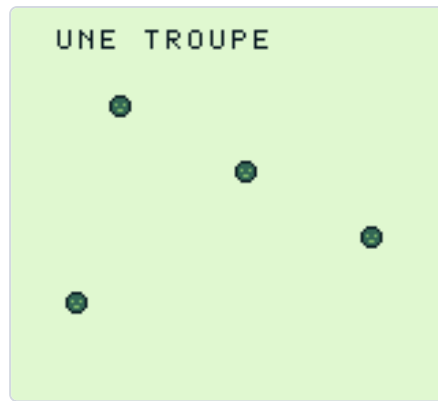
    return 0;
}
```

3. Ce qui se passe, étape par étape

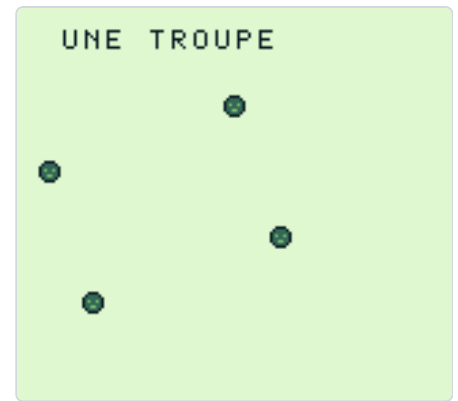
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



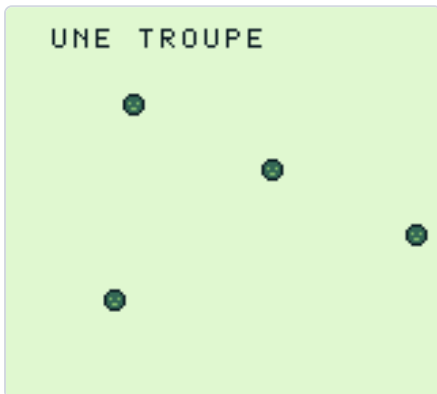
la toute première image — la console vient de démarrer — image 1



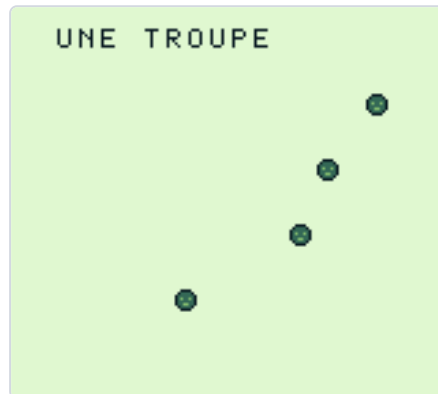
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Quatre boules qui traversent l'écran, chacune à sa vitesse.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ les quatre lutins portent la tuile dessinée
- ✓ ils sont sur quatre lignes différentes
- ✓ tous avancent (30, 60, 191, 120)
- ✓ et chacun à sa vitesse (30, 60, 191, 120)
- ✓ le titre est écrit

Ensuite : **84. Le décor qui défile** — chapitre 13 : Ce qui va ensemble. · `gameboy3` — écrire une cartouche Game Boy en C++